

CSE 420: Compiler Design

You must answer all the questions. **Answer all the subparts of a question together.**
Theoretical questions should be answered in a *precise, concise, and to-the-point manner*.

Name:	ID:	Section:
-------	-----	----------

Question 1 [CO5] [2 marks]

When we are generating Three Address Code for Control flow statements, **why** do we calculate labels as inherited attributes?

Question 2 [CO5] [3 marks]

Why are temporaries and local data kept at the end of the activation record? **Which components** of a callee's activation record is the callee responsible for and **why**?

Question 3 [CO4] [1 + 1 = 2 marks]

When a compiler converts a class member function to a global function, **how** does the scope of variables change? Specifically, **how** are class-level fields accessed differently from local variables inside the method?

Question 4 [CO5] [1.5 + 1.5 + 2 = 5 marks]

Observe the following code and answer the questions afterwards.

Note: It is not necessary to show any symbol table.

<pre>struct P { char grade; int marks; char section[6]; };</pre>	<pre>struct Q { int courseId; P result; char room[5]; };</pre>	<pre>struct R { char title[8]; Q course; int semester; };</pre>	<pre>struct S { int studentId; R info; char university[12]; };</pre>
--	--	---	--

Now, Suppose the following variable is declared:

S studentRecord;

Assume that:

- **int** occupies 4 bytes and **char** occupies 1 byte.
- **@studentRecord** denotes the base address of the object **studentRecord**.

- Determine** the total offset of the field accessed in the following statement relative to **@studentRecord**:
y = studentRecord.info.course.result.section[2];
- Write** the equivalent three-address code for the statement above in part a.
- Describe** how subclasses in inheritance are represented in memory layout during compilation. Explain with an example why a stack might be necessary to implement this.

Question 5 [CO3] [3 + 3 = 6 marks]

- The following CFG and SDD are for translating composite boolean conditions involving AND connective. **Find** any error in the SDD and **write** the correct SDD.

$B \rightarrow B_1 \parallel B_2 \{ B_1.true = newLabel(); B_1.false = B.false; B_2.true = B.true; B_2.false = B.false; \\ B.code = B_1.code \parallel gen(B_1.true ":") \parallel B_2.code; \}$
--

- Why** does short-circuit evaluation improve the generated three-address code for Boolean expressions? Give an **example**.

Question 6 [CO4] [5 marks]

Using the grammar and semantic rules given, **draw** the parse tree and **write** the generated Three Address Code outputs for the following assignment statement: **y[seq[x + k]] = p;**

You must label the parse tree with the appropriate attributes.

*Note: Assume that both **y** and **seq** are **integer** arrays of **length 15** and each **integer** occupies **4 bytes**. Variables **x**, **k** and **p** are **integers**. The **gen()** function generates strings and the **Temp()** class creates a temporary variable which is '**t**' with a subscript 1, 2, 3 and so on.*

SDT:

```
S → id = E ; { gen(top.get(id.lexeme) '=' E.addr); }
  | L = E ; { gen(L.array.base '[' L.addr ']' '=' E.addr); }
E → E1 + E2 { E.addr = new Temp(); gen(E.addr '=' E1.addr '+' E2.addr); }
  | id { E.addr = top.get(id.lexeme); }
  | L { E.addr = new Temp(); gen(E.addr '=' L.array.base '[' L.addr ']'); }
L → id [ E ] { L.array = top.get(id.lexeme); L.type = L.array.type.elem; L.addr = new Temp();
               gen(L.addr '=' E.addr '*' L.type.width); }
  | L1 [ E ] { L.array = L1.array; L.type = L1.type.elem; t = new Temp(); L.addr = new Temp();
               gen(t '=' E.addr '*' L.type.width); gen(L.addr '=' L1.addr '+' t); }
```

(Scoring breakdown ⇒ Parse tree : 2 + Annotation : 1 + Three Address Code : 2 = 5 marks)

Question 7 [CO4] [5 + 2 = 7 marks]

- a. Using the grammar and semantic rules given, draw the parse tree and the symbol table for the following declarations. You must label the parse tree with the appropriate attributes:

Code Snippet:

```
int i;
record {
    int[5][10] p;
} z;
```

SDT:

```
P → { tab = new SymbolTable(); off = 500; stTab = new Stack(); stOff = new Stack(); }
    D
D → T id ; { tab.insert(id.lexeme, { type: T.t, width: T.w, offset: off }); off = off + T.w; }
    D
D → ε
T → B { C.bt = B.t; C.bw = B.w; }
    C { T.t = C.t; T.w = C.w; }
T → record '{ { stTab.push(tab); stOff.push(off); tab = new SymbolTable(); off = 0; }
    D }' { T.t = tab; T.w = off * 2; tab = stTab.pop(); off = stOff.pop(); }
B → int { B.t = i; B.w = 3; }
B → float { B.t = f; B.w = 6; }
C → [ num ] { C1.bt = C.bt; C1.bw = C.bw }
    C1 { C.t = array(num.value, C1.t); C.w = num.value * C1.w + 5 }
C → ε { C.t = C.bt; C.w = C.bw; }
```

(Scoring breakdown ⇒ Parse tree : 2 + Annotation : 2 + Symbol Table : 1 = 5 marks)

- b. Explain why syntax-directed translation computes array width using recursive multiplication of dimensions and element width.
-